

The Singular Value Decomposition (SVD)

Lecture Notes

Zamir Hussain
Department of Mathematics
University of Wah

January 12, 2026

1 Motivation: Why We Need SVD

In previous lectures, we have explored several matrix factorizations like LU and QR decomposition. You might ask: why do we need another one? The motivation for the Singular Value Decomposition (SVD) arises from the limitations of standard linear algebra concepts when dealing with real-world, noisy data.

While theoretical concepts like “rank” or “invertibility” are binary (a matrix is either one or the other), practical applications demand a more nuanced view. The SVD provides this nuance and is driven by three powerful applications:

1. **Numerical Stability and Sensitivity Analysis:** Real-world matrices are rarely truly singular, but they can be “almost singular.” SVD helps us quantify *how close* a matrix is to being singular, which is crucial for understanding how small errors in input data can lead to huge errors in the output.
2. **Data Compression and Low-Rank Approximation:** Many large data matrices (like user-movie ratings) are “almost low rank.” SVD provides the *optimal* way to factor such matrices into much smaller, manageable pieces, leading to massive gains in storage and computational speed.
3. **Dimensionality Reduction and PCA:** SVD is the mathematical engine behind **Principal Component Analysis (PCA)**, allowing us to find the best-fitting low-dimensional subspaces for complex, high-dimensional data points.

2 The Challenge with “Almost Singular” Matrices

Let’s explore the first motivation more deeply. Consider solving a simple equation $ax = b$. If a is a very small number, say $a = 0.002$, and our measurement of b has a tiny error, the solution x becomes highly unstable.

Example Problem

[Sensitivity to Small Perturbations] Suppose the true equation is $0.002x = 1$, for which the correct solution is $x = 500$. Now, imagine our measurement of a is not exact and has an error of ± 0.001 .

✔ Step-by-Step Solution

1. **Case 1 (Upper Error Bound):** Let $a = 0.002 + 0.001 = 0.003$.

$$0.003x = 1 \implies x = \frac{1}{0.003} \approx 333.3$$

2. **Case 2 (Lower Error Bound):** Let $a = 0.002 - 0.001 = 0.001$.

$$0.001x = 1 \implies x = \frac{1}{0.001} = 1000$$

💡 Key Idea

A tiny uncertainty in the coefficient a (from 0.001 to 0.003) leads to a massive uncertainty in the solution x , which ranges from 333 to 1000!

In higher dimensions, this problem is less obvious but more dangerous. A matrix A in $A\mathbf{x} = \mathbf{b}$ might have seemingly normal numbers, but if its columns are nearly linearly dependent, it is “almost singular.” SVD gives us a way to measure this instability through its singular values.

3 Defining the Singular Value Decomposition (SVD)

Theorem

[Singular Value Decomposition] Any $m \times n$ matrix A with $\text{rank}(A) = r$ can be factored into:

$$A = U\Sigma V^T$$

where:

- U is an $m \times m$ **orthogonal matrix** ($U^T U = I_m$). Its columns, $\mathbf{u}_i$, are the **left singular vectors**. The first r columns, $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$, form an orthonormal basis for the column space of A .
- Σ is an $m \times n$ rectangular **diagonal matrix**. Its diagonal entries, σ_i , are the **singular values** of A . They are non-negative and ordered by convention: $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, with $\sigma_{r+1} = \dots = 0$.
- V is an $n \times n$ **orthogonal matrix** ($V^T V = I_n$). Its columns, $\mathbf{v}_i$, are the **right singular vectors**. The first r columns, $\{\mathbf{v}_1, \dots, \mathbf{v}_r\}$, form an orthonormal basis for the row space of A .

Placeholder for Diagram:

A figure showing the shapes of A , U , Σ , and V^T for an $m \times n$ matrix, e.g., where $m > n$. Visually show that Σ has a diagonal block of singular values and is padded with zeros.

Figure 1: Structure of the SVD factorization.

Key Idea

[Geometric Interpretation] The SVD tells us that any linear transformation A can be broken down into three simple operations:

1. A **rotation** or reflection in the domain space (given by V^T).
2. A **scaling** along the new coordinate axes (given by Σ).
3. A **rotation** or reflection in the co-domain space (given by U).

The fundamental relationship connecting these components is:

$$A\mathbf{v}_i = \sigma_i \mathbf{u}_i \quad \text{for } i = 1, \dots, r$$

This means that A maps the i -th right singular vector to the i -th left singular vector, scaled by the i -th singular value.

4 How to Compute the SVD

The SVD is deeply connected to the eigenvalues and eigenvectors of two related symmetric matrices: $A^T A$ and AA^T .

4.1 Procedure for finding U, Σ, V

1. Compute V and Σ from $A^T A$:

- Calculate the symmetric $n \times n$ matrix $B = A^T A$.
- Find the eigenvalues λ_i and corresponding orthonormal eigenvectors of B .
- The **right singular vectors** ($\mathbf{v}_i$) are these eigenvectors. Form the matrix $V = [\mathbf{v}_1 | \dots | \mathbf{v}_n]$.
- The **singular values** (σ_i) are the square roots of the eigenvalues: $\sigma_i = \sqrt{\lambda_i}$. Form the diagonal matrix Σ with these values, ordered from largest to smallest.

2. Compute U :

- Use the formula $A\mathbf{v}_i = \sigma_i \mathbf{u}_i$ to find the first r **left singular vectors** ($\mathbf{u}_i$):

$$\mathbf{u}_i = \frac{1}{\sigma_i} A\mathbf{v}_i \quad (\text{for all } \sigma_i > 0)$$

- If more columns are needed for U (i.e., if $m > r$), find the remaining columns by extending $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$ to a full orthonormal basis for $\mathbb{R}^m$. This can be done by finding a basis for the null space of A^T and using the Gram-Schmidt process.

4.2 Step-by-Step Calculation Example

Example Problem

Find the Singular Value Decomposition of the matrix $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}$.

✓ Step-by-Step Solution

Step 1: Compute $A^T A$

$$A^T A = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

Step 2: Find Eigenvalues and Eigenvectors of $A^T A$ to get Σ and V

- **Eigenvalues:** The characteristic equation is $\det(A^T A - \lambda I) = 0$.

$$(2 - \lambda)^2 - 1^2 = 0 \implies (2 - \lambda) = \pm 1$$

The eigenvalues are $\lambda_1 = 3$ and $\lambda_2 = 1$.

- **Singular Values (Σ):**

$$\sigma_1 = \sqrt{3}, \quad \sigma_2 = \sqrt{1} = 1$$

Since A is 3×2 , Σ is also a 3×2 matrix:

$$\Sigma = \begin{pmatrix} \sqrt{3} & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$$

- **Eigenvectors (V):**

- For $\lambda_1 = 3$: $(A^T A - 3I)\mathbf{v}_1 = \begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \mathbf{0}$. This gives $x = y$. The normalized eigenvector is $\mathbf{v}_1 = \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix}$.
- For $\lambda_2 = 1$: $(A^T A - 1I)\mathbf{v}_2 = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \mathbf{0}$. This gives $x = -y$. The normalized eigenvector is $\mathbf{v}_2 = \begin{pmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \end{pmatrix}$.

$$V = \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{pmatrix}$$

Step 3: Compute the Left Singular Vectors (U)

- For $\sigma_1 = \sqrt{3}$:

$$\mathbf{u}_1 = \frac{1}{\sigma_1} A \mathbf{v}_1 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} = \frac{1}{\sqrt{6}} \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}$$

- For $\sigma_2 = 1$:

$$\mathbf{u}_2 = \frac{1}{\sigma_2} A \mathbf{v}_2 = \frac{1}{1} \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ -1 \\ 1 \end{pmatrix}$$

- We need a third vector $\mathbf{u}_3$ for the 3×3 matrix U that is orthogonal to both $\mathbf{u}_1$ and $\mathbf{u}_2$. We can find it by finding a basis vector for the null space of A^T . Solving $A^T \mathbf{x} = \mathbf{0}$:

$$\begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies x + z = 0, x + y = 0 \implies \mathbf{x} = \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix}$$

Normalizing this gives $\mathbf{u}_3 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix}$.

Finally, we assemble the full U matrix:

$$U = \begin{pmatrix} 2/\sqrt{6} & 0 & 1/\sqrt{3} \\ 1/\sqrt{6} & -1/\sqrt{2} & -1/\sqrt{3} \\ 1/\sqrt{6} & 1/\sqrt{2} & -1/\sqrt{3} \end{pmatrix}$$

The complete SVD is $A = U\Sigma V^T$ with the matrices found above.

5 Application: Low-Rank Approximation and Data Compression

This is where the SVD shines for large datasets. The **Eckart-Young Theorem** states that the best rank- k approximation to a matrix A (in the sense of minimizing error) is given by the truncated SVD.

Definition

[Truncated SVD] To get a rank- k approximation, we keep only the first k singular values and the corresponding first k columns of U and V .

$$A_k = U_k \Sigma_k V_k^T$$

where U_k is an $m \times k$ matrix, Σ_k is a $k \times k$ diagonal matrix, and V_k^T is a $k \times n$ matrix. This is the optimal $A \approx BC$ factorization.

Key Idea

[Netflix Example] Imagine a matrix A of user ratings: 10 million users ($m = 10^7$) by 10,000 movies ($n = 10^4$).

- **Full Matrix Storage:** $m \times n = 10^{11}$ entries. At 4 bytes per entry, this is **400 GB**.
- **Low-Rank Approximation (assume rank $k = 10$):**
 - We store U_{10} ($10^7 \times 10$), Σ_{10} (10×10), and V_{10}^T (10×10^4).
 - **Total entries:** $(10^7 \times 10) + (10 \times 10) + (10 \times 10^4) \approx 10^8$.
 - **SVD Storage:** 10^8 entries at 4 bytes each is **0.4 GB**.

This represents a **1000x reduction in storage** and a similar reduction in computation time for operations like matrix-vector products, making analysis feasible.

Key Idea

[Connection to PCA] The SVD is the mathematical engine behind Principal Component Analysis (PCA). When we have a set of high-dimensional data points, PCA aims to find a lower-dimensional subspace that best fits the data.

- The right singular vectors (v_i in V) are the **principal components**, representing the directions of maximum variance in the data.

- The left singular vectors (U) combined with the singular values (Σ) give the coordinates of the data points in this new, reduced-dimensional basis.

Finding the top k principal components is equivalent to finding the best rank- k SVD approximation.

6 Summary

- The SVD ($A = U\Sigma V^T$) is a powerful factorization that provides a nuanced view of a matrix's properties, going beyond binary concepts like rank.
- The **singular values** (σ_i) in Σ are the heart of the SVD. Small singular values indicate the matrix is close to being rank-deficient or numerically unstable.
- The ratio σ_1/σ_r (the **condition number**) is a direct measure of a matrix's sensitivity to numerical errors.
- SVD provides the **best low-rank approximation** to a matrix, making it the premier tool for data compression, noise reduction, and dimensionality reduction (PCA).

7 Demonstration: Polynomial Fitting and Cross-Validation

Theoretical concepts like SVD and least squares come to life when applied to real problems like model fitting. In this section, we provide a practical demonstration of polynomial regression using both MATLAB and Python. The goal is to fit a polynomial to a set of noisy data points and use Cross-Validation (CV) to determine the optimal polynomial degree.

💡 Key Idea

The problem: A simple, low-degree polynomial (**underfitting**) will fail to capture the underlying trend in the data. A complex, high-degree polynomial (**overfitting**) will fit the noise in the training data perfectly but will generalize poorly to new, unseen data. Cross-validation is a technique to estimate a model's performance on unseen data and helps us find the sweet spot between underfitting and overfitting.

The task is to find a polynomial $p(x) = c_d x^d + \dots + c_1 x + c_0$ that best fits a given set of data points (x_i, y_i) . This is a linear least-squares problem, which can be formulated as solving $Ac = y$ for the coefficient vector c , where A is the Vandermonde matrix.

7.1 MATLAB Implementation: `poly_fitting.m`

MATLAB's built-in functions make polynomial fitting and evaluation straightforward. Here's a script that demonstrates fitting polynomials of different degrees and evaluating their performance using a validation set.

```
</> MATLAB Code: poly_fitting.m
```

```
% poly_fitting.m: Regression + CV Demo
```

```
% 1. Generate noisy sample data
```

```

rng(42); % for reproducibility
x_train = linspace(0, 1, 50)';
y_true = sin(2*pi*x_train);
y_train = y_true + 0.2 * randn(size(x_train));

x_val = linspace(0, 1, 100)';
y_val_true = sin(2*pi*x_val);
y_val_noise = y_val_true + 0.2*randn(size(x_val));

% 2. Loop through polynomial degrees
max_degree = 15;
train_errors = zeros(max_degree, 1);
val_errors = zeros(max_degree, 1);

figure;
hold on;
plot(x_train, y_train, 'bo', 'DisplayName', 'Training Data');
plot(x_val, y_val_true, 'g-', 'LineWidth', 2, 'DisplayName', 'True');
xlabel('x');
ylabel('y');
title('Polynomial Fitting of Various Degrees');
grid on;
legend('show');

for d = 1:max_degree
    % 3. Fit polynomial using polyfit
    % Solves the least-squares problem  $A*c = y$ 
    coeffs = polyfit(x_train, y_train, d);

    % 4. Predict on training and validation sets
    y_train_pred = polyval(coeffs, x_train);
    y_val_pred = polyval(coeffs, x_val);

    % 5. Calculate Mean Squared Error (MSE)
    train_errors(d) = mean((y_train - y_train_pred).^2);
    val_errors(d) = mean((y_val_noise - y_val_pred).^2);

    % Plot fits for a few selected degrees
    if ismember(d, [1, 3, 10])
        plot(x_val, y_val_pred, '--', 'LineWidth', 1.5, ...
            'DisplayName', sprintf('Degree %d Fit', d));
    end
end
hold off;

% 6. Plot the error curves (CV curve)
figure;
plot(1:max_degree, train_errors, 'b-o', 'DisplayName', 'Training Error');
hold on;
plot(1:max_degree, val_errors, 'r-s', 'DisplayName', 'Validation Error');

```

```

xlabel('Polynomial Degree (d)');
ylabel('Mean Squared Error (MSE)');
title('Cross-Validation: Finding the Optimal Degree');
legend('show');
grid on;
[min_val_error, best_d] = min(val_errors);
fprintf('Optimal polynomial degree found: %d\n', best_d);
xline(best_d, '--k', sprintf('Best d = %d', best_d));

```

Code Explanation:

1. **Data Generation:** We create a set of '(x, y)' points based on a true sine wave, then add random noise. We also create a separate, larger validation set to test our model's generalization performance.
2. **Looping through Degrees:** The code iterates through polynomial degrees from 1 to 15.
3. **polyfit** This is the core function. It takes the 'x', 'y' data and the degree 'd', and it solves the linear least-squares problem to find the best-fitting polynomial coefficients.
4. **polyval** This function uses the fitted coefficients to predict 'y' values for a new set of 'x' points.
5. **Error Calculation:** We calculate the Mean Squared Error (MSE) for both the training and validation sets.
6. **Plotting:** The first plot shows how different polynomial degrees fit the data. The second plot shows the training vs. validation error, which is the classic "cross-validation curve" used to identify the optimal model complexity. The degree with the lowest validation error is considered the best model.

7.2 Python Implementation: `poly_fitting.py`

The same logic can be implemented in Python using libraries like 'NumPy' for numerical operations and 'scikit-learn' for a more structured approach to model fitting.

Python Code: `poly_fitting.py`

```

# poly_fitting.py: Regression + CV demo
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# 1. Generate noisy sample data
np.random.seed(42)
n_samples = 50

```

```

X_train = np.sort(np.random.rand(n_samples))
y_true = np.sin(2 * np.pi * X_train)
y_train = y_true + 0.2 * np.random.randn(n_samples)

X_val = np.linspace(0, 1, 100)
y_val_true = np.sin(2 * np.pi * X_val)
y_val_noise = y_val_true + 0.2 * np.random.randn(100)

# Reshape for scikit-learn
X_train = X_train[:, np.newaxis]
X_val = X_val[:, np.newaxis]

# 2. Loop through degrees
max_degree = 15
train_errors, val_errors = [], []

plt.figure(figsize=(12, 6))
plt.scatter(X_train, y_train, color='blue', label='Training Data')
plt.plot(X_val, y_val_true, color='green', linewidth=2, label='True')

for d in range(1, max_degree + 1):
    # 3. Create and fit the polynomial model
    # A pipeline is a clean way to chain transformations
    model = make_pipeline(PolynomialFeatures(d),
                          LinearRegression())
    model.fit(X_train, y_train)

    # 4. Predict on training and validation sets
    y_train_pred = model.predict(X_train)
    y_val_pred = model.predict(X_val)

    # 5. Calculate MSE
    train_errors.append(mean_squared_error(y_train, y_train_pred))
    val_errors.append(mean_squared_error(y_val_noise, y_val_pred))

    if d in [1, 3, 10]:
        plt.plot(X_val, y_val_pred, '--',
                 label=f'Degree {d} Fit')

plt.title('Polynomial Fitting of Various Degrees')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()

# 6. Plot the error curves
best_d = np.argmin(val_errors) + 1
plt.figure(figsize=(10, 5))
plt.plot(range(1, max_degree + 1), train_errors, 'o-', color='blue')

```

```
plt.plot(range(1, max_degree + 1), val_errors, 's-', color='red', label='Validation Error')
plt.axvline(best_d, linestyle='--', color='k', label=f'Best Degree {best_d}')
plt.title('Cross-Validation Curve')
plt.xlabel('Polynomial Degree')
plt.ylabel('Mean Squared Error (MSE)')
plt.legend()
plt.grid(True)
plt.xticks(range(1, max_degree + 1))
plt.show()

print(f"Optimal polynomial degree found: {best_d}")
```

Code Explanation:

1. **Libraries:** 'NumPy' is used for numerical array handling, 'matplotlib' for plotting, and 'scikit-learn' for its machine learning pipeline tools.
2. **Data Generation:** Similar to MATLAB, we create synthetic training and validation datasets. Note the use of '`[:, np.newaxis]`' to reshape the data into the format expected by scikit-learn (a 2D array).
3. **make_pipeline** This scikit-learn utility is very powerful. It creates a model that first applies 'PolynomialFeatures' (which generates the columns of the Vandermonde matrix: $1, x, x^2, \dots$) and then fits a 'LinearRegression' model on these new features. This handles the entire least-squares process.
4. **model.fit() and model.predict()** These are the standard methods in scikit-learn for training a model and making predictions.
5. **mean_squared_error** A convenient function for calculating MSE.
6. **Plotting:** The logic is identical to the MATLAB version, culminating in a CV curve that helps us identify the optimal polynomial degree where the validation error is minimized.

💡 Key Idea

[Results Analysis] In both the MATLAB and Python demonstrations, the cross-validation plot will typically show:

- The **training error** continuously decreases as the polynomial degree increases. A more complex model can always fit the training data better.
- The **validation error** initially decreases, hits a minimum (the "sweet spot"), and then starts to increase again as the model begins to overfit the noise in the training data.

The optimal degree is found at the minimum of the validation error curve. This illustrates the fundamental bias-variance trade-off in machine learning and statistical modeling.